Cooper Taffe

Numerical PDEs Homework #2

Problem 1: Verify HW0 Code

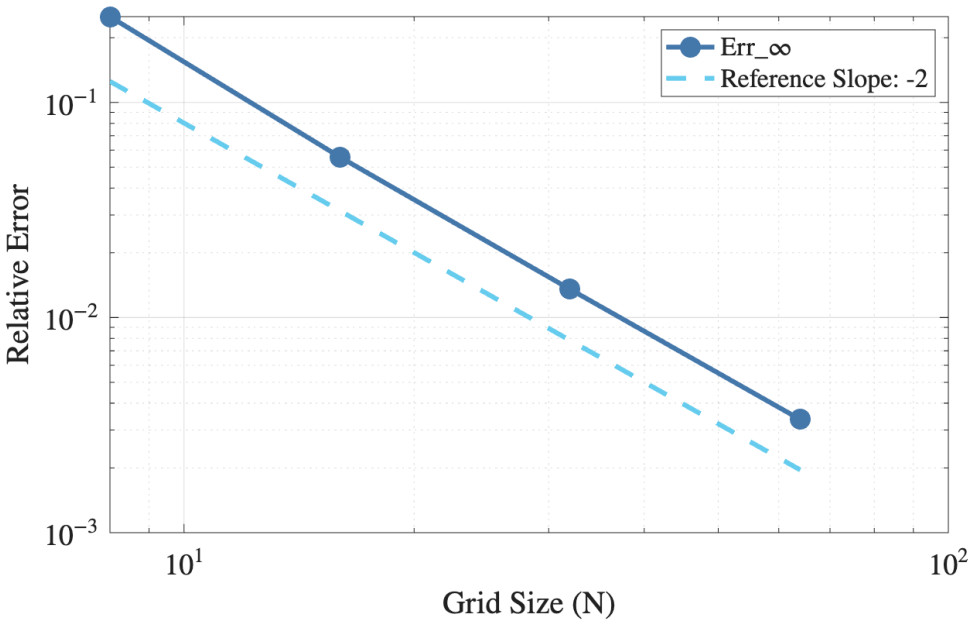


Figure 1: With the new manufactured solution, my HW0 code still gives us second order convergence.

Problem 2: Poisson Equation with Immersed Boundary

a) After just adding the immersed boundary conditions, my code does not have second order convergence. I held the grid spaces with an N=32, then making the IB spacing N=8 the error was 2.1074 and the elapsed time was 0.698297 seconds. Then enlarging the IB spacing to N=128 the error was similar at 2.1036 but the elapsed time is now 19.503406 seconds, which is a lot longer. We kind of talked about this in class and a way to make it faster is to just let the delta function be 0 when the inputs are a reasonable distance away.

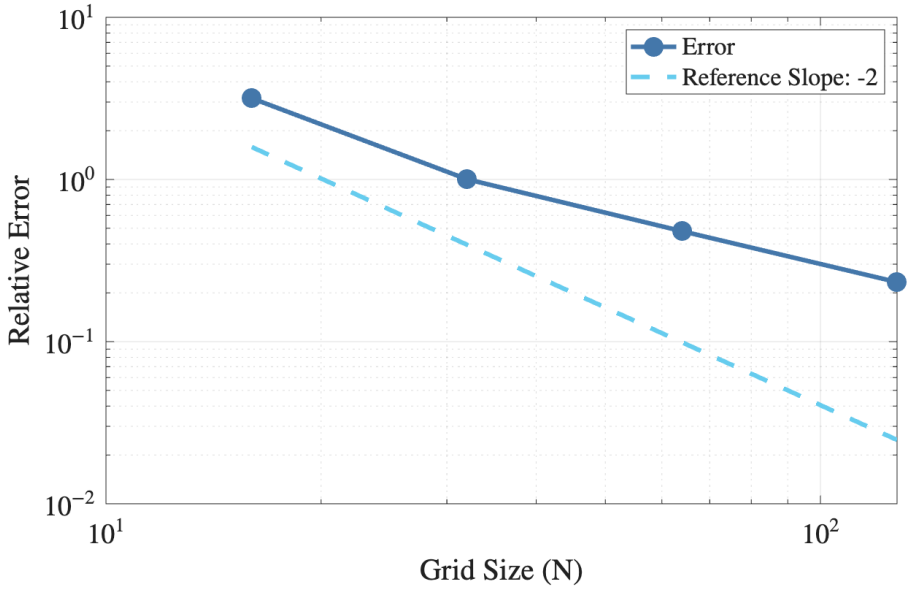


Figure 2: Error using N=16,32,64,128. We can see this doesn't exactly follow the second order convergence check and we know that the immersed boundary has an error of $\Delta x^{1.5}$

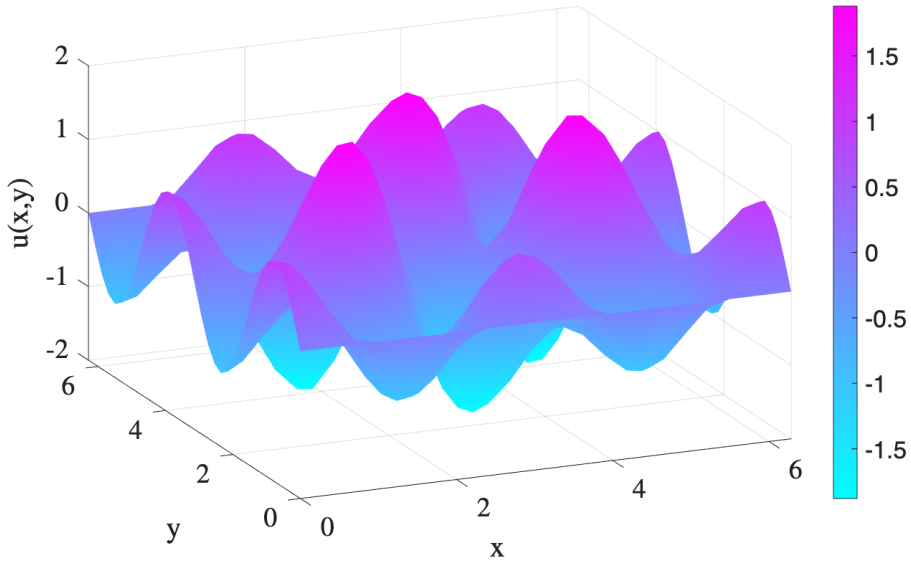


Figure 3: Solution of u(x,y) using finite difference methods and an immersed boundary. The solution is found using $f(x, y) = -8\cos(2x)\sin(2y)$

Problem 3:

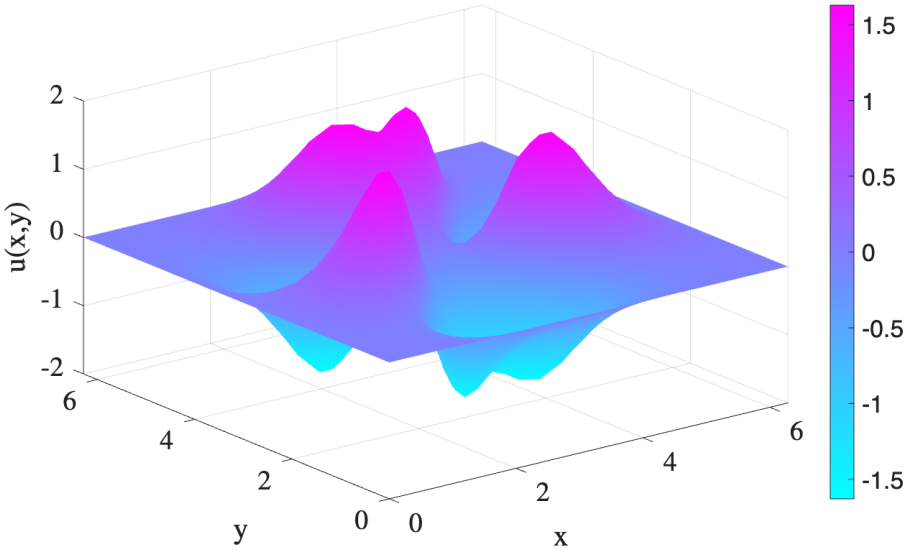


Figure 4: Solution of u(x,y) for f(x,y) = 0 and dirichlet boundary conditions. The solver uses finite difference methods and an immersed boundary.

Problem 4: Charlie's Rule

I decided to try and speed up my code by only evaluating the delta function if it would be above a certain tolerance. Inside both the applyJ and applyS functions I used an if statement to see if delta would be greater than a given tolerance and then summed such $Q\delta$ and $\phi\delta$ terms, otherwise I just went to the next iteration. It seemed like it made the first couple iterations of Ns faster but at N=128 it didn't seem to go a lot faster. I also played with how low I could set the tolerance before the error was affected. I got to 0.01 without the error graph changing but at .1 the method was slower and the error was worse.

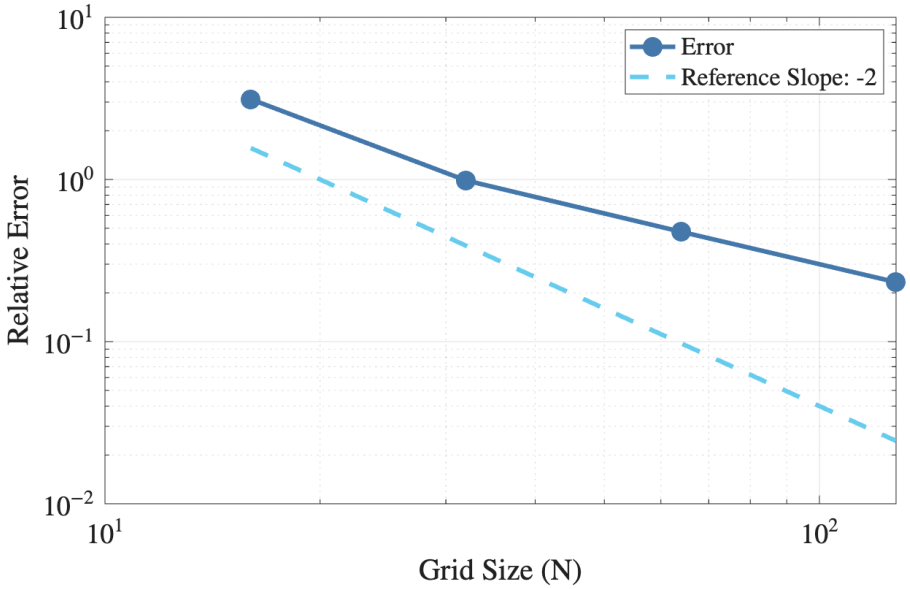


Figure 5: Error using N=16,32,64,128 and only evaluating the delta function when greater than 0.01.